

Variable Projection for Improved Training of Tensor Neural Networks

Vutichart Buranasiri

May 2026

Abstract

Deep neural networks have become state-of-the-art for various tasks such as image compression, but dense fully connected layers may require an extremely large number of parameters when input data is naturally multidimensional. Tensor neural networks (t-NNs) provide a solution to this problem by replacing standard matrix-vector products with well defined, matrix-mimetic tensor-tensor products, which preserve the inherent image structure and reduce the number of trainable weights by an order of magnitude. This research project studies whether variable projection (VarPro), originally an optimization technique for separable nonlinear least-squares problems, can further improve the training of a t-NN autoencoder on the MNIST dataset. By exploiting the fact that the final decoder layer of a t-NN is affine-linear, VarPro separates the final layer into a linear and nonlinear part with respect to the parameter weights. Then, the optimal linear part of the final layer can be solved analytically by least squares, leaving the nonlinear part to be trained by standard gradient descent methods. The VarPro training scheme is implemented for a t-NN autoencoder and compared to standard training with Adam across several training set sizes. Results indicate that VarPro improves test reconstruction loss for small and medium dataset sizes but incurs additional least-squares overhead and becomes less advantageous when the training set is large.

1 Introduction

Deep neural networks (DNNs) have become the standard in recent years for image recognition, language modeling, scientific computing, and many other high-dimensional learning tasks. The inherent success of DNNs is attributed to their ability to learn deep, nonlinear compositions to approximate highly complicated functions. However, this flexibility comes at the cost of using dense linear layers, which often contain millions of parameter weights, thus requiring significant training time. Other additional problems, such as overfitting, may occur when the data set is small relative to the number of parameters.

One such example where this problem arises is with image data. A standard grayscale MNIST image $I \in \mathbb{R}^{28 \times 28}$ has shape 28×28 , and a standard fully connected deep neural network vectorizes this image as input into a vector in $I_{\text{vec}} \in \mathbb{R}^{784}$. A dense, square weight matrix acting on this vector would thus require $784^2 = 614,656$ weights. Thus, an $n \times n$ image would require weight matrices with parameters on the order of n^4 per layer under the standard DNN architecture. Additionally, the vectorization of images as input discards the two-dimensional, spatial structure of the image and treats all pixels as unrelated coordinates.

Tensor neural networks (t-NNs), which were introduced by Newman, Horesh, Avron, and Kilmer [1], provide a more structured alternative. Instead of vectorizing an image as input, a t-NN represents each image input as a third-order tensor $I \in \mathbb{R}^{28 \times 1 \times 28}$, and applies a weight tensor $\mathcal{W}$ at

each layer of the t-NN using the $\star_M$ tensor-tensor product [1]. This product is matrix-mimetic: it retains analogues of matrix multiplication, transposition, and orthogonality, among other desirable properties, while operating directly on multi-way data. Specifically For MNIST data, a t-NN layer can reduce the number of parameters by roughly a factor of 28 compared with a corresponding dense matrix layer seen in standard DNN architecture [1].

Additionally, stochastic gradient descent and adaptive variants such as Adam have become the standard for training deep neural networks. In particular, solvers such as Adam update all weights through mini-batch gradients [2], and the iterations have been shown empirically to lead to convergence to global minimizers that can generalize well. While these methods are flexible and scalable, they often require careful choices of learning rate, batch size, scheduling, initialization, and regularization. They also treat every parameter block in essentially the same way, even when part of the network enters the loss linearly and could be solved for directly. In that case, gradient descent spends iterations approximating a quantity that has a closed-form least-squares solution. This observation motivates using variable projection (VarPro) for training.

VarPro, originally developed by Golub and Pereyra for separable nonlinear least-squares problems [3], eliminates variables that enter a model linearly. In neural network training, Newman, Ruthotto, Hart, and van Bloemen Waanders showed that VarPro can be used when the last layer of a DNN is affine-linear: the earlier layers form a nonlinear feature map, while the last layer can be separated into a linear and nonlinear part based on those features [4].

In this project, we aim to combine the parameter-efficient t-NN architecture with the VarPro training scheme, and compare this training method with standard gradient descent updates via Adam. We apply this idea to a t-NN autoencoder on the MNIST dataset comprising of 28×28 grayscale images of handwritten digits between 0-9. The t-NN autoencoder compresses each image through a low-dimensional latent representation and then reconstructs the image. We train under the VarPro scheme and standard gradient descent with Adam and compare the results using Mean Squared Error (MSE) as our reconstruction loss. The main contributions of this project are:

- an implementation of the VarPro training scheme for a t-NN autoencoder on MNIST.
- a derivation of the least-squares systems solve under VarPro induced by the $\star_M$ product in the t-NN architecture.
- an empirical comparison between standard Adam training and VarPro training of t-NN autoencoders across training set sizes ranging from 1,000, 5,000, 10,000, 20,000, and 50,000 images.
- an analysis of when the closed-form decoder update helps and when its overhead or instability limits performance.

2 Related Work

2.1 Tensor products and tensor neural networks

Historically, tensor methods have been designed and implemented to represent and process multi-way data without needing to vectorize or flatten it into vectors or matrices. This is especially relevant for data modalities such as images or videos, where the spatial coordinates have meaningful and important multidimensional structure. The t-NN framework used in this project is built on a family of tensor-tensor products that preserve many matrix-like properties while acting directly on third-order tensors.

The first matrix-mimetic tensor product formulation came from the t-product framework, devised by Kilmer and Martin [5]. The t-product provides a way to multiply third-order tensors by transforming using the Discrete Fourier Transform along the tube dimension, multiplying frontal slices, and transforming back. Kernfeld, Kilmer, and Aeron generalized this idea by allowing tensor-tensor products to be defined through arbitrary invertible linear transforms [6]. Kilmer, Horesh, Avron, and Newman further developed this transform-based tensor algebra as a tool for representation and compression of multiway data [7].

Newman, Horesh, Avron, and Kilmer then incorporate this matrix-mimetic tensor algebra within the context of deep learning, defining tensor neural networks (t-NNs) [1]. A t-NN replaces the dense matrix multiplication in a fully connected layer with a tensor-tensor product, allowing image data to remain in tensor form rather than being vectorized. This reduces parameter count by an order of magnitude and preserves multidimensional structure. Their work also connects t-NNs to stable neural network architectures, building on ideas from stable and Hamiltonian neural networks [8]. This body of work project uses the t-NN framework as the model class but focuses specifically on improving the training of a t-NN autoencoder through variable projection.

2.2 Variable projection

Variable projection (VarPro) is an optimization method used more generally for problems whose variables separate into linear and nonlinear parts. It was originally introduced by Golub and Pereyra for nonlinear least-squares problems whose variables separate [3]. Later work developed practical implementations and extensions of the method for nonlinear least-squares problems [9], and related VarPro ideas have been applied to optimized dynamic mode decomposition [10]. More recently, variable projection has been adapted to deep neural network training. Newman, Ruthotto, Hart, and van Bloemen Waanders observe that neural networks with an affine last layer can be written in the separable form

$$G(y, W, \theta) = WF(y, \theta),$$

where $F(y, \theta)$ is the nonlinear feature extractor formed by the earlier layers and W is the final affine layer [4]. In this setting, VarPro solves for the optimal final layer for the current features and trains the remaining network weights through the reduced problem. Their work develops a Gauss-Newton VarPro method and extends the idea beyond squared loss to objectives such as cross-entropy loss for classification.

This project sits at the intersection of these two lines of work. It uses the tensor neural network architecture of Newman et al.[1], but instead of training all weights by Adam, it applies the separable-optimization idea from VarPro to the final decoder layer. The task is also different from most prior neural-network VarPro examples: rather than supervised classification or surrogate modeling, the loss is unsupervised reconstruction error for an MNIST autoencoder.

3 Proposed Approach and Algorithm

3.1 Tensor notation and the $\star_M$ product

We first introduce the relevant notation for third-order tensors, which align with work from Kernfeld, Kilmer, and Aeron [6]. Let $\mathcal{A} \in \mathbb{R}^{m \times \ell \times n}$ be a third-order tensor. While intuitively it seems simpler to denote entries as indexed by three coordinates, it is often more useful to describe the tensor through certain specific decompositions of lower-dimensional slices and fibers. We establish the following ways of decomposing such tensors:

- A *frontal slice* of the tensor $\mathcal{A}$ fixes the third index and varies the first two indices: $\mathcal{A}^{(k)} = \mathcal{A}(:, :, k) \in \mathbb{R}^{m \times \ell}$. These are 2-dimensional matrices and are crucial for defining the matrix-mimetic $\star_M$ product.
- A *lateral slice* fixes the second index and varies the first and third: $\tilde{\mathcal{A}}_j = \mathcal{A}(:, j, :) \in \mathbb{R}^{m \times 1 \times n}$. In the t-NN architecture, each MNIST image is stored as a lateral slice as input.
- A *tube* or mode-3 fiber fixes the first two indices and varies the third: $\mathbf{a}_{ij} = \mathcal{A}(i, j, :) \in \mathbb{R}^{1 \times 1 \times n}$. The orthogonal transform M , relevant to the $\star_M$ product, acts along these tubes. They can be viewed as the analog of scalars in the tensor setting.

Thus, we can interpret third-order tensors in two ways: As matrices whose entries are tubes, or as a stack of matrices (frontal slices) along the third dimension. Both of these interpretations are crucial in our t-NN implementation. Visualizations of the above tensor decompositions are provided in Figure 1 below.

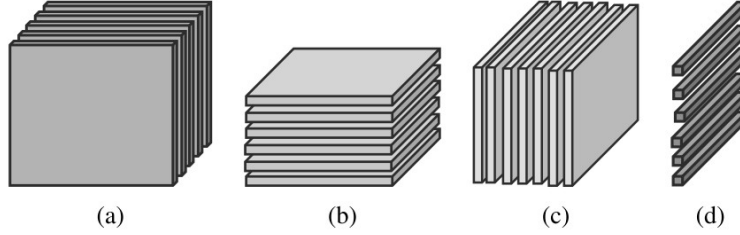


Fig. 1. (a) Frontal, (b) horizontal, and (c) lateral slices of a 3-rd order tensor. The lateral slices are also referred to as *oriented matrices*. (d) A lateral slice as a vector of tube fibers.

Figure 1: Frontal, horizontal, and lateral slices of a third-order tensor, along with a lateral slice represented as a vector of tube fibers. Figure adapted from Kernfeld, Kilmer, and Aeron [6].

Let $\mathcal{A} \in \mathbb{R}^{m_1 \times \ell \times n}$ and $\mathcal{B} \in \mathbb{R}^{\ell \times m_2 \times n}$, and let $M \in \mathbb{R}^{n \times n}$ be an invertible transform. We first introduce, for a tensor $\mathcal{X} \in \mathbb{R}^{a \times b \times n}$ and a matrix $M \in \mathbb{R}^{p \times n}$, the mode-3 product

$$\mathcal{Y} = \mathcal{X} \times_3 M \in \mathbb{R}^{a \times b \times p}$$

defined by

$$\mathcal{Y}_{ijk} = \sum_{s=1}^n \mathcal{X}_{ijs} M_{ks}, \quad i = 1, \dots, a, \quad j = 1, \dots, b, \quad k = 1, \dots, p.$$

Applying the mode-3 product is equivalent to multiplying each tube $\mathcal{X}(i, j, :)$ by M . We can then define the transform versions of $\mathcal{A}$ and $\mathcal{B}$ under M , denoted $\hat{\mathcal{A}}$ and $\hat{\mathcal{B}}$, by:

$$\hat{\mathcal{A}} = \mathcal{A} \times_3 M, \quad \hat{\mathcal{B}} = \mathcal{B} \times_3 M,$$

where $\times_3$ denotes the mode-3 product defined above. The $\star_M$ product between two tensors $\mathcal{A}$ and $\mathcal{B}$ is defined as:

$$\mathcal{A} \star_M \mathcal{B} = (\hat{\mathcal{A}} \triangle \hat{\mathcal{B}}) \times_3 M^{-1},$$

where $\triangle$ denotes facewise multiplication, given by

$$(\hat{\mathcal{A}} \triangle \hat{\mathcal{B}})^{(k)} = \hat{\mathcal{A}}^{(k)} \hat{\mathcal{B}}^{(k)}, \quad k = 1, \dots, n.$$

Defined for each frontal slice. Thus the $\star_M$ product reduces one tensor multiplication to n independent matrix multiplications in the transform domain. In the experiments below, we set M to be the Discrete Cosine Transform (DCT) matrix of size 28. This is chosen because the DCT matrix is real-valued and computation of the $\star_M$ product under this choice of M is computationally efficient.

3.2 Tensor neural network layers

A tensor neural network layer replaces the dense matrix multiplication in a standard neural network with the $\star_M$ tensor-tensor product. For MNIST, instead of vectorizing the image as input into the form $I_{\text{vec}} \in \mathbb{R}^{784}$, the image is kept as a lateral slice

$$\tilde{\mathcal{Y}} \in \mathbb{R}^{28 \times 1 \times 28}.$$

The first and third modes store the two image coordinates, while the middle mode has length one for a single MNIST image. In implementation however, we note that mini-batches are formed by stacking these lateral slices along the middle mode. Thus a batch of N inputs to layer r is represented as

$$\mathcal{Z}_{r-1}^{\text{batch}} \in \mathbb{R}^{m_{r-1} \times N \times n},$$

where m_{r-1} is the layer width and $n = 28$ is the tube dimension for MNIST. For a single image input, this reduces to $\mathcal{Z}_{r-1} \in \mathbb{R}^{m_{r-1} \times 1 \times n}$. A t-NN layer with output width m_r has weight tensor

$$\mathcal{W}_r \in \mathbb{R}^{m_r \times m_{r-1} \times n}$$

and bias tensor

$$\mathcal{B}_r \in \mathbb{R}^{m_r \times 1 \times n},$$

where the bias is broadcast across the N samples. The layer output is

$$\mathcal{Z}_r = \sigma(\mathcal{W}_r \star_M \mathcal{Z}_{r-1} + \mathcal{B}_r) \in \mathbb{R}^{m_r \times N \times n},$$

where σ is the nonlinear activation function applied entrywise. In this project, we use $\sigma = \tanh$, the hyperbolic tangent function, as our nonlinear activation. Note that for a dataset containing $n \times n$ images, a standard DNN has weight matrix containing parameters on the order of $n^2 \times n^2 = n^4$. Each t-NN weight tensor $\mathcal{W}_r$ has parameters on the order of $n \times n \times n = n^3$, an order of magnitude less than a standard DNN model. Additionally, we note that the same matrix-mimetic structure also makes backpropagation through a t-NN layer closely analogous to backpropagation through a standard DNN layer. For an orthogonal transform M , Newman et al. [1] give the directional derivative identities

$$\frac{\partial}{\partial \mathcal{Z}} [\mathcal{W} \star_M \mathcal{Z}] = \mathcal{W}^\top \star_M \partial \mathcal{Z}$$

and

$$\frac{\partial}{\partial \mathcal{W}} [\mathcal{W} \star_M \mathcal{Z}] = \partial \mathcal{W} \star_M \mathcal{Z}^\top,$$

where $\partial \mathcal{Z}$ and $\partial \mathcal{W}$ denote perturbations with the same sizes as $\mathcal{Z}$ and $\mathcal{W}$. These formulas have the same form as the corresponding matrix-product derivatives, with matrix multiplication replaced by $\star_M$ and transposes replaced by tensor transposes. Consequently, t-NN weights can be trained with standard stochastic gradient optimizers such as Adam.

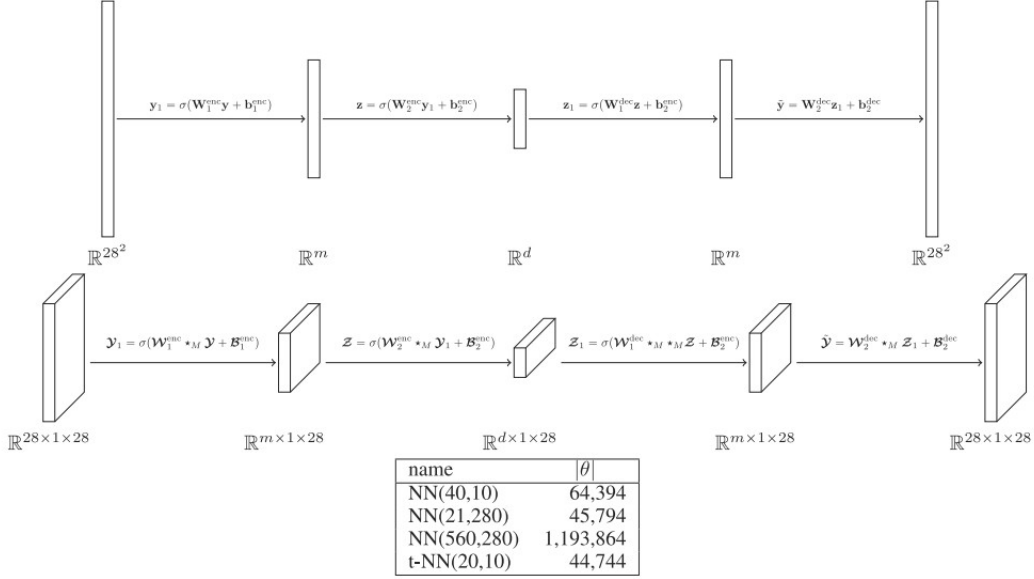


Figure 2: Comparison of a standard vectorized autoencoder and a tensor neural network autoencoder. The t-NN preserves the MNIST image as a tensor and replaces matrix products with $\star_M$ products. Figure adapted from Newman et al. [1].

3.3 Variable projection background

Variable projection applies when the trainable variables split into two groups: one group enters the model linearly, while the other group enters through a nonlinear feature map. Suppose that we are given N training samples. Let

$$Y \in \mathbb{R}^{q \times N}$$

be the matrix of target outputs, with one training example per column and output dimension q . For fixed nonlinear parameters θ , let

$$F(\theta) \in \mathbb{R}^{d \times N}$$

be the feature matrix produced by the earlier layers of the network, where d is the feature dimension. A final linear layer with weights

$$W \in \mathbb{R}^{q \times d}$$

then predicts

$$\hat{Y} = WF(\theta) \in \mathbb{R}^{q \times N}.$$

The corresponding separable least-squares problem is

$$\min_{W, \theta} \|WF(\theta) - Y\|_F^2,$$

where W is the linear variable and θ controls the nonlinear feature map. If θ is fixed, then $F(\theta)$ is just a known $d \times N$ matrix, so the only unknown in the objective is the $q \times d$ matrix W . The problem therefore becomes an ordinary linear least-squares problem:

$$\min_{W \in \mathbb{R}^{q \times d}} \|WF(\theta) - Y\|_F^2.$$

When this least-squares problem can be solved with the Moore-Penrose pseudoinverse, using methods such as Singular Value Decomposition (SVD). The optimal linear variable is

$$W^*(\theta) = YF(\theta)^\dagger,$$

where

$$F(\theta)^\dagger \in \mathbb{R}^{N \times d}$$

and therefore $W^*(\theta) \in \mathbb{R}^{q \times d}$. Substituting $W^*(\theta)$ back into the objective removes W from the optimization and gives the reduced problem

$$\min_{\theta} \|W^*(\theta)F(\theta) - Y\|_F^2.$$

Instead of taking gradient steps in both W and θ , VarPro repeatedly solves for the globally optimal linear weights W for the current features and uses gradient-based training via Adam on the non-linear parameters θ . We formulate the t -NN equivalent representation of VarPro in the following sections.

3.4 Autoencoder architecture

As with any other autoencoder framework, the t-NN has an encoder and decoder representation of the form:

$$f(y) = f_{\text{dec}}(f_{\text{enc}}(y)).$$

Where the encoder compresses an image into a latent tensor representation, and the decoder maps this representation back to the original image shape. In our experiments, the architecture used is a 2-layer t-NN, where each layer compresses the autoencoder width by the following dimensions.

$$28 \longrightarrow 20 \longrightarrow 10 \longrightarrow 20 \longrightarrow 28,$$

The bottleneck width is 10, the hidden decoder width is 20, and all intermediate activations are tanh activation functions. The final decoder layer is affine-linear, with no activation function. This is what makes the last layer of the model is separable. The training objective is mean squared reconstruction error:

$$\min_{\theta_{\text{enc}}, \theta_{\text{dec}}} \frac{1}{N} \sum_{i=1}^N \left\| f_{\text{dec}}(f_{\text{enc}}(\tilde{\mathcal{Y}}_i)) - \tilde{\mathcal{Y}}_i \right\|_F^2.$$

We aim to reproduce reconstructed MNIST digits using our t-NN autoencoder architecture, evaluating based on the above MSE loss, training time and training stability.

3.5 Closed-form decoder update

We now derive the how the VarPro training scheme from section 3.2 is implemented with t-NN architecture. Let θ denote all weights except the final decoder layer, and let

$$\Phi(\mathcal{Y}, \theta) \in \mathbb{R}^{20 \times N \times 28}$$

be the tensor of decoder features immediately before the final layer, computed over all N training images. We denote the target tensor

$$\mathcal{Y} \in \mathbb{R}^{28 \times N \times 28}.$$

Given that the final decoder layer has weight tensor $\mathcal{W} \in \mathbb{R}^{28 \times 20 \times 28}$ and bias tensor $\mathcal{B} \in \mathbb{R}^{28 \times 1 \times 28}$. The reconstruction has the tensor form

$$\mathcal{Y}_{\text{rec}} = \mathcal{W} \star_M \Phi(\mathcal{Y}, \theta) + \mathcal{B}.$$

Thus the training problem can be written as the separable nonlinear least-squares problem

$$\min_{\theta, \mathcal{W}, \mathcal{B}} \|\mathcal{W} \star_M \Phi(\mathcal{Y}, \theta) + \mathcal{B} - \mathcal{Y}\|_F^2.$$

Note that this objective is nonlinear in θ , since the feature tensor $\Phi(\mathcal{Y}, \theta)$ is produced by the encoder and first decoder layer. However for fixed θ , $\Phi(\mathcal{Y}, \theta)$ is known and the problem becomes affine-linear in the final-layer variables $(\mathcal{W}, \mathcal{B})$:

$$\min_{\mathcal{W}, \mathcal{B}} \|\mathcal{W} \star_M \Phi + \mathcal{B} - \mathcal{Y}\|_F^2.$$

Note that we can apply the DCT transform along the third mode, which converts the tensor least-squares problem into independent matrix least-squares problems, one for each frontal slice. Because the DCT matrix M is orthogonal, this transform preserves the Frobenius norm of the residual. Let the tensors in the transform domain be denoted

$$\widehat{\Phi} = \Phi \times_3 M, \quad \widehat{\mathcal{Y}} = \mathcal{Y} \times_3 M, \quad \widehat{\mathcal{W}} = \mathcal{W} \times_3 M, \quad \widehat{\mathcal{B}} = \mathcal{B} \times_3 M.$$

In the transform domain, the fixed- θ final-layer problem is therefore

$$\min_{\widehat{\mathcal{W}}, \widehat{\mathcal{B}}} \|\widehat{\mathcal{W}} \triangle \widehat{\Phi} + \widehat{\mathcal{B}} - \widehat{\mathcal{Y}}\|_F^2,$$

where $\triangle$ denotes facewise matrix multiplication. Since M is the 28×28 DCT matrix, there are 28 transformed frontal slices. For each $k = 1, \dots, 28$, the decoupled final-layer problem in the transform domain becomes

$$\min_{\widehat{W}^{(k)}, \widehat{b}^{(k)}} \|\widehat{W}^{(k)} \widehat{\Phi}^{(k)} + \widehat{b}^{(k)} \mathbf{1}^\top - \widehat{\mathcal{Y}}^{(k)}\|_F^2.$$

Here

$$\widehat{W}^{(k)} \in \mathbb{R}^{28 \times 20}, \quad \widehat{\Phi}^{(k)} \in \mathbb{R}^{20 \times N}, \quad \widehat{b}^{(k)} \in \mathbb{R}^{28 \times 1}, \quad \widehat{\mathcal{Y}}^{(k)} \in \mathbb{R}^{28 \times N}.$$

Where $\widehat{b}^{(k)} \in \mathbb{R}^{28 \times 1}$ denotes the frontal slice of $\widehat{\mathcal{B}}$, which is a vector. The original tensor problem therefore separates into 28 independent matrix problems, each solving for the 28×20 weight matrix and 28 bias entries per frontal slice. By augmenting the feature matrix:

$$\widetilde{\Phi}^{(k)} = \begin{bmatrix} \widehat{\Phi}^{(k)} \\ \mathbf{1}^\top \end{bmatrix} \in \mathbb{R}^{21 \times N}, \quad X^{(k)} = \begin{bmatrix} \widehat{W}^{(k)} & \widehat{b}^{(k)} \end{bmatrix} \in \mathbb{R}^{28 \times 21}.$$

We obtain the linear least-squares problem

$$\min_{X^{(k)} \in \mathbb{R}^{28 \times 21}} \|X^{(k)} \widetilde{\Phi}^{(k)} - \widehat{\mathcal{Y}}^{(k)}\|_F^2.$$

Thus, the optimal augmented final layer is obtained via the pseudoinverse:

$$X^{(k)*} = \widehat{\mathcal{Y}}^{(k)} \left(\widetilde{\Phi}^{(k)} \right)^\dagger.$$

This solve is performed independently for each slice $k = 1, \dots, 28$, and the resulting weights are then transformed back to the original tensor domain. Substituting these optimal final-layer weights back into the objective gives the reduced nonlinear problem

$$\min_{\theta} \|\mathcal{W}^*(\theta) \star_M \Phi(\mathcal{Y}, \theta) + \mathcal{B}^*(\theta) - \mathcal{Y}\|_F^2,$$

where $(\mathcal{W}^*(\theta), \mathcal{B}^*(\theta))$ denotes the decoder layer obtained from the 28 least-squares solves above. In our implementation, the remaining parameters are updated with mini-batch Adam while the final decoder layer is kept fixed at its current VarPro solution.

3.6 Training algorithm

The training procedure alternates between a closed-form final-layer update and standard gradient updates on the remaining weights. We introduce a warm-up period before beginning VarPro training, because at initialization the feature map Φ is essentially random and the least-squares solution can be unstable.

Algorithm 1 VarPro training for the t-NN autoencoder

- 1: Initialize all network weights.
 - 2: **for** epochs $1, \dots, n_{\text{warmup}}$ **do**
 - 3: Train all weights with mini-batch Adam.
 - 4: **end for**
 - 5: **for** remaining epochs **do**
 - 6: Run the encoder and first decoder layer on all training data to form $\Phi(\mathcal{Y}, \theta)$.
 - 7: Apply the DCT transform along the third tensor mode to obtain $\hat{\Phi}$ and $\hat{\mathcal{Y}}$.
 - 8: **for** $k = 1, \dots, 28$ **do**
 - 9: Form $\tilde{\Phi}^{(k)} = [\hat{\Phi}^{(k)}; \mathbf{1}^\top]$.
 - 10: Solve $X^{(k)*} = \hat{\mathcal{Y}}^{(k)}(\tilde{\Phi}^{(k)})^\dagger$.
 - 11: Extract $\widehat{W}^{(k)*}$ and $\widehat{b}^{(k)*}$ from $X^{(k)*}$.
 - 12: **end for**
 - 13: Transform the final-layer weights and biases back to tensor space.
 - 14: Train only the encoder and intermediate decoder weights with mini-batch Adam, keeping the final decoder layer fixed at its VarPro value.
 - 15: **end for**
-

4 Results

All experiments use the MNIST handwritten digit data set [11]. We fix the validation set, which contains 10,000 images, and the training set size is varied over

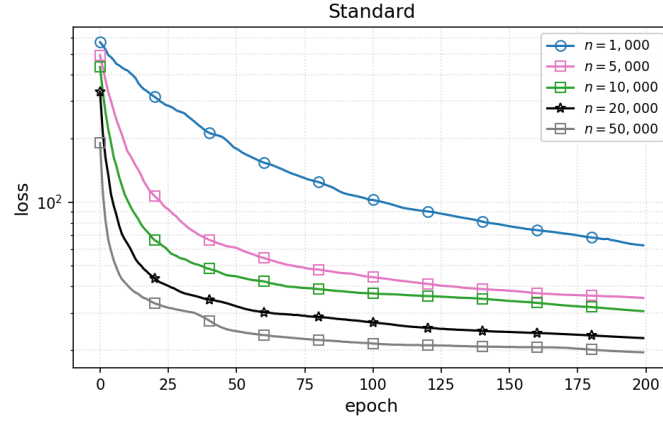
$$N \in \{1,000, 5,000, 10,000, 20,000, 50,000\}.$$

The model is a t-NN autoencoder with widths $28 \rightarrow 20 \rightarrow 10 \rightarrow 20 \rightarrow 28$, DCT transform M , tanh activations, and 44,744 total parameters. Standard training uses Adam on all weights [2]. VarPro training uses closed-form least squares for the final decoder layer and Adam for the remaining weights. In both implementations, Adam uses learning rate 10^{-3} , weight decay 0, mini-batch size 32, and 200 total training epochs. The Adam momentum parameters are PyTorch’s defaults

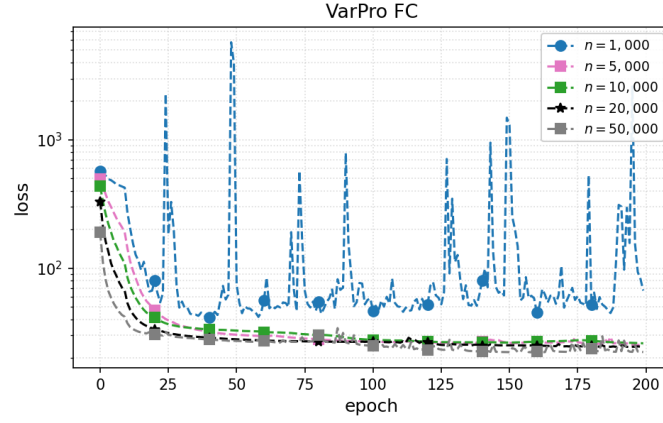
$(\beta_1, \beta_2) = (0.9, 0.999)$. A StepLR scheduler is included with step size 100 and decay factor $\gamma = 1$, so the learning rate remains 10^{-3} throughout training. For VarPro, the first 10 epochs are warm-up epochs using Adam before least-squares decoder updates begin, and gradients on the Adam-trained parameters are clipped to norm at most 1. Both methods are evaluated by MSE test loss at the checkpoint with lowest validation loss.

Table 1: Test reconstruction loss and total training time. The loss values are reported as MSE multiplied by 10^3 .

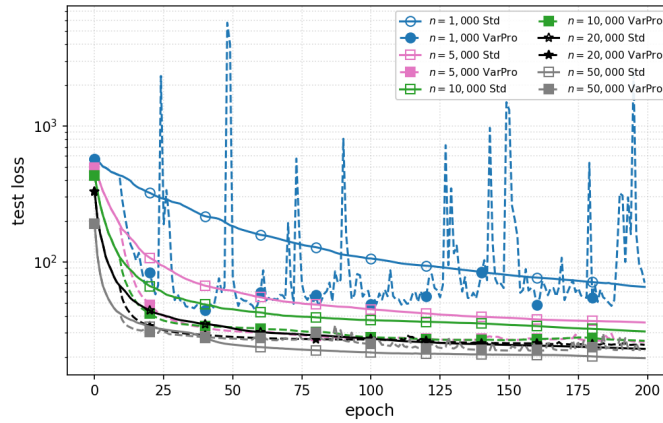
N_{train}	MSE test loss ($\times 10^{-3}$)		Training time (s)	
	Standard Adam	VarPro	Standard Adam	VarPro
1,000	65.32	44.10	96.9	160.9
5,000	35.73	25.71	467.1	787.0
10,000	30.70	26.20	976.0	1405.3
20,000	22.85	24.44	1740.6	2869.7
50,000	19.52	22.15	4874.4	7134.0



(a) Standard Adam across all training set sizes.

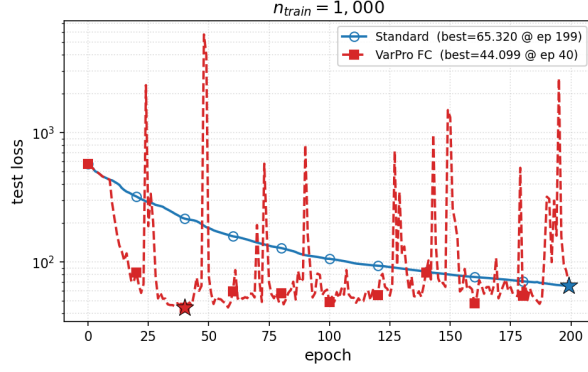


(b) VarPro across all training set sizes.

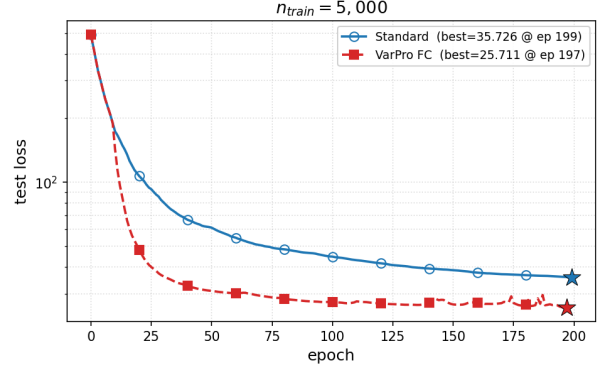


(c) Combined comparison of standard Adam and VarPro.

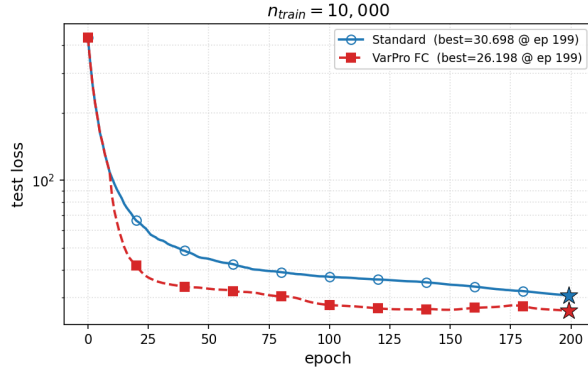
Figure 3: Aggregate loss plots.



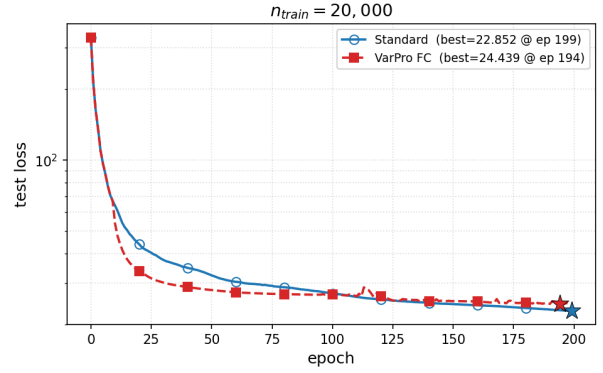
(a) $N_{\text{train}} = 1,000$.



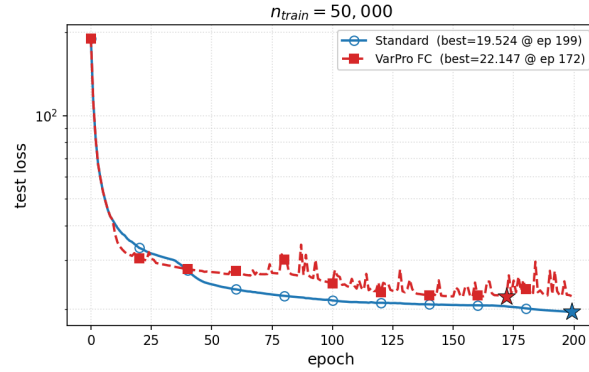
(b) $N_{\text{train}} = 5,000$.



(c) $N_{\text{train}} = 10,000$.



(d) $N_{\text{train}} = 20,000$.



(e) $N_{\text{train}} = 50,000$.

Figure 4: Per-training-size loss convergence plots comparing standard Adam and VarPro.

5 Discussion

The results from Figure 4, show that VarPro is most useful when the data set is small or medium sized. For $N = 1,000$, 5,000, and 10,000, VarPro achieves lower test reconstruction loss than standard Adam training. This supports the central motivation of the method: when part of the model is linear, solving that part exactly can remove a source of optimization error. The final decoder layer is always globally optimal for the current nonlinear features, so the optimizer does not need to spend gradient steps learning that layer.

We analyze the behaviour at $N = 1,000$, which is an especially interesting case. The VarPro loss curve is significantly less smooth than the standard Adam curve, but the final test loss is better, which is achieved at epoch 40. This suggests that the closed-form decoder solve can compensate for limited data despite noisier training dynamics. A likely explanation is that small changes in the encoder can substantially change the feature matrix Φ , causing the least-squares decoder solution $W^*(\theta)$ to jump significantly between epochs. The resulting feedback can produce oscillations as seen in Figure 4 (a). Nevertheless, because each decoder is optimal for the current features, the method still finds a representation with lower reconstruction error.

For larger data sets, the advantage of VarPro decreases. As seen from Figure 4 (d) and (e), at $N = 20,000$ and $N = 50,000$, we observe that standard Adam achieves lower test loss. This may be attributed to the fact that with more data, the stochastic gradients used by Adam have lower variance, and the optimizer can train all layers to a more effective degree. In this data regime, the exact final-layer solve provides only marginal benefits while still adding computational cost. We note that in these experiments, VarPro training takes about 1.5–1.7 times as long as standard training because each epoch includes 28 least-squares solves over the full training set.

We note that the tensor structure is crucial to making VarPro computationally reasonable. Without the $\star_M$ transform-domain decoupling, the final-layer tensor least-squares problem would be more difficult to solve directly. However, under the $\star_M$ product, it becomes 28 independent matrix least-squares problems, one for each transformed frontal slice. This is both mathematically clean and naturally parallelizable. However, the current implementation solves these systems sequentially rather than parallelizing over the 28 frontal slices. The reported VarPro training times therefore reflect an unoptimized implementation, and a parallel implementation could reduce the overhead of the closed-form decoder update. This reflects one limitation of our experiments.

A further limitation in this work is that the least-squares systems are unregularized, so the small-data regime may be sensitive to conditioning. If the feature matrix $\tilde{\Phi}^{(k)}$ is nearly rank deficient, the pseudoinverse solution can change sharply from one epoch to the next. This may explain some of the significant number of oscillations seen in the VarPro loss curves. A Tikhonov-regularized solve, given by the equation,

$$X_\lambda^{(k)*} = \hat{\mathcal{Y}}^{(k)} \left(\tilde{\Phi}^{(k)} \right)^\top \left(\tilde{\Phi}^{(k)} \left(\tilde{\Phi}^{(k)} \right)^\top + \lambda I \right)^{-1},$$

could further stabilize the decoder update. Finally, the experiments have been restricted to reconstruction tasks on the MNIST dataset, on fully connected t-NN layers. While MNIST is a useful benchmark for isolating the optimization effect, it remains a small grayscale dataset with a relatively simple structure. More complex, multidimensional image data sets such as the CIFAR-10 dataset may require more advanced or deep t-NN architectures in which the setup in our experiments may not generalize well, and the balance between VarPro accuracy gains and least-squares overhead may change in those settings.

6 Conclusion

In this project, we investigate whether variable projection is capable of improving the training of tensor neural networks. The proposed method exploits the separable structure of a t-NN autoencoder by solving for the optimal final decoder layer exactly via solving a linear least squares problem and training the remaining nonlinear weights with Adam. The $\star_M$ product makes this feasible and efficient by reducing the tensor least-squares problem to independent matrix problems in the transform domain. Empirically, we observe that VarPro improves MNIST reconstruction loss for training set sizes up to 10,000, but standard Adam performs better for larger data sets. The results suggest that VarPro is most valuable when the exact solution of the linear block reduces optimization error enough to justify its computational overhead.

There are several natural directions for future work. One algorithmic extension would be to add regularization to the closed-form decoder update, such as Tikhonov regularization to the linear least-squares formulation, to improve conditioning when the feature matrix is nearly rank-deficient. Another direction is to study whether the same VarPro implementation remains useful for other t-NN layer types, such as stable Hamiltonian t-NN layers and residual layers [1]. Furthermore, the relationship between t-NN autoencoder depth and VarPro performance would be an interesting area of further analysis. We also hope to extend experiments to more challenging, multidimensional image data sets such as the CIFAR-10 dataset. It would also be useful to further optimize the VarPro implementation by parallelizing the per-slice least-squares solves. Finally, a broader extension would be to adapt the current VarPro approach to classification tasks by combining t-NN architecture with the Gauss-Newton VarPro (GNvpro) training scheme for cross-entropy loss [4].

References

- [1] E. Newman, L. Horesh, H. Avron, and M. E. Kilmer, “Stable tensor neural networks for efficient deep learning,” *Frontiers in Big Data*, vol. 7, p. 1363978, 2024. DOI: 10.3389/fdata.2024.1363978.
- [2] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *3rd International Conference on Learning Representations (ICLR)*, 2015.
- [3] G. H. Golub and V. Pereyra, “The differentiation of pseudo-inverses and nonlinear least squares problems whose variables separate,” *SIAM Journal on Numerical Analysis*, vol. 10, no. 2, pp. 413–432, 1973. DOI: 10.1137/0710036.
- [4] E. Newman, L. Ruthotto, J. Hart, and B. van Bloemen Waanders, “Train like a (var)pro: Efficient training of neural networks with variable projection,” *arXiv preprint arXiv:2007.13171*, 2021. arXiv: 2007.13171 [cs.LG].
- [5] M. E. Kilmer and C. D. Martin, “Factorization strategies for third-order tensors,” *Linear Algebra and its Applications*, vol. 435, no. 3, pp. 641–658, 2011. DOI: 10.1016/j.laa.2010.09.020.
- [6] E. Kernfeld, M. Kilmer, and S. Aeron, “Tensor–tensor products with invertible linear transforms,” *Linear Algebra and its Applications*, vol. 485, pp. 545–570, 2015. DOI: 10.1016/j.laa.2015.07.021.
- [7] M. E. Kilmer, L. Horesh, H. Avron, and E. Newman, “Tensor-tensor algebra for optimal representation and compression of multiway data,” *Proceedings of the National Academy of Sciences*, vol. 118, no. 28, e2015851118, 2021. DOI: 10.1073/pnas.2015851118.

- [8] E. Haber and L. Ruthotto, “Stable architectures for deep neural networks,” *Inverse Problems*, vol. 34, no. 1, p. 014004, 2018. DOI: 10.1088/1361-6420/aa9a90.
- [9] D. P. O’Leary and B. W. Rust, “Variable projection for nonlinear least squares problems,” *Computational Optimization and Applications*, vol. 54, no. 3, pp. 579–593, 2013. DOI: 10.1007/s10589-012-9492-9.
- [10] T. Askham and J. N. Kutz, “Variable projection methods for an optimized dynamic mode decomposition,” *SIAM Journal on Applied Dynamical Systems*, vol. 17, no. 1, pp. 380–416, 2018. DOI: 10.1137/M1124176.
- [11] Y. LeCun, C. Cortes, and C. J. C. Burges, “The MNIST database of handwritten digits,” <http://yann.lecun.com/exdb/mnist/>, 2010.

A Reconstruction Examples

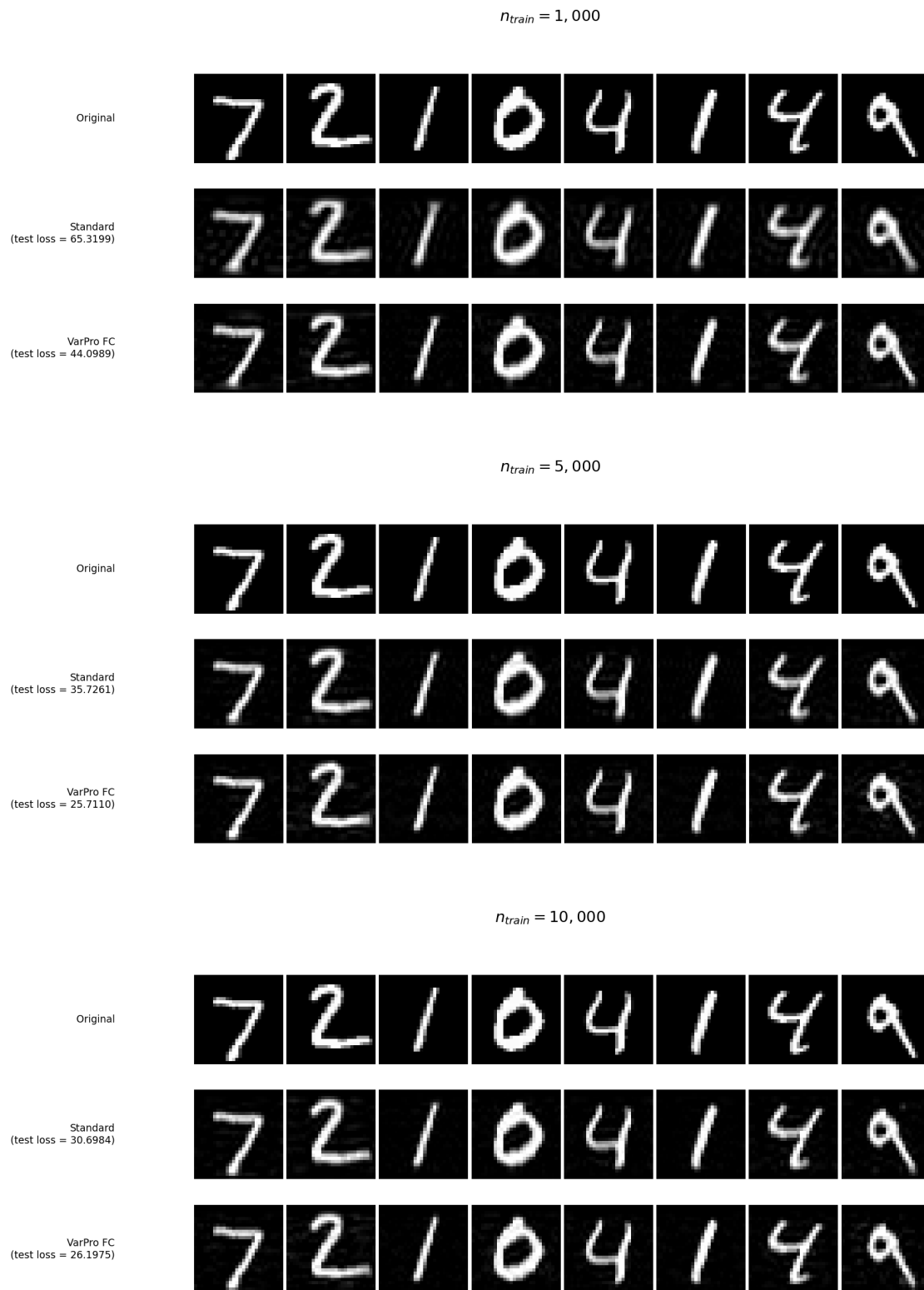


Figure 5: Example reconstructions for small and medium training set sizes.

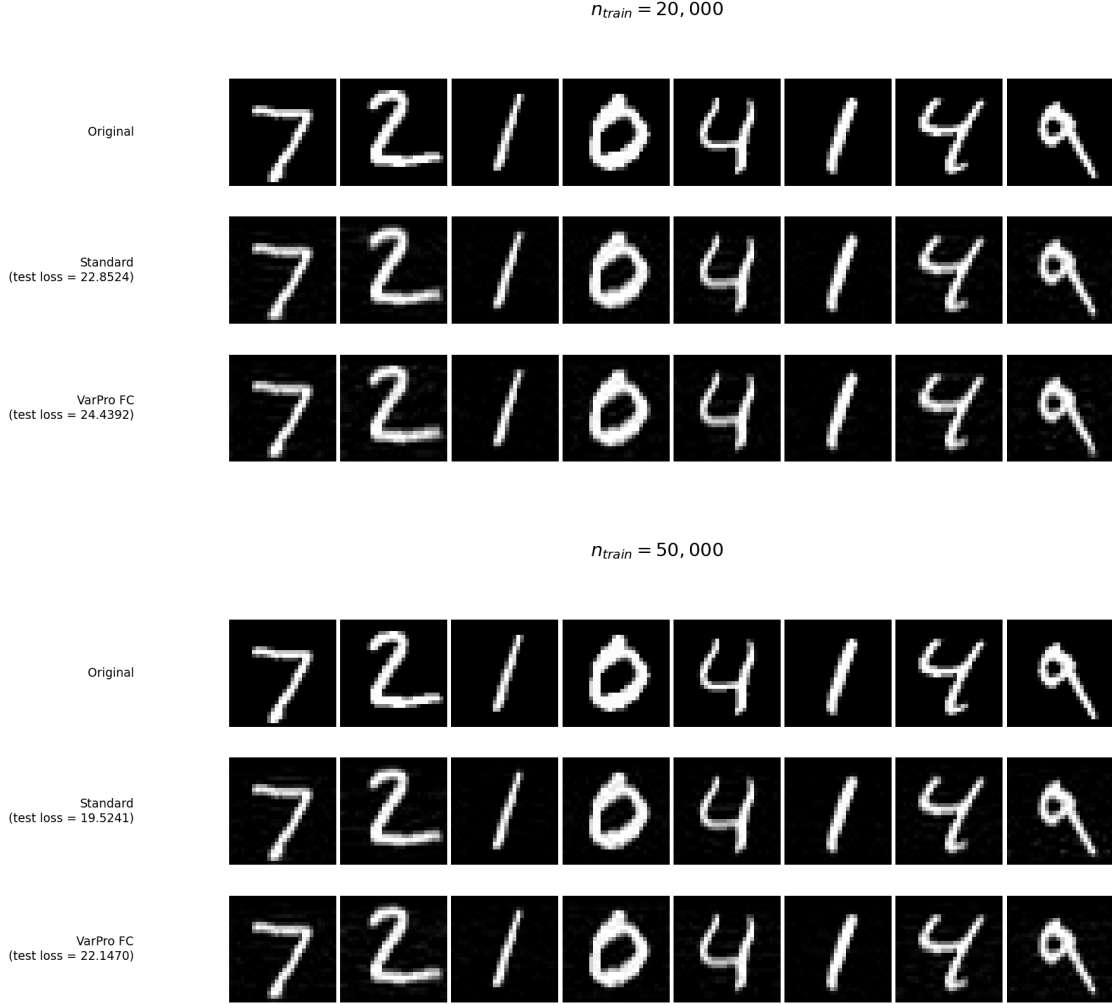


Figure 6: Example reconstructions for larger training set sizes.

B Implementation Details

Code is available at: <https://github.com/vburan01/tnn-VarPro-main.git>. The main implementation files are:

- `ex_mnist_tensor_standard_regression.py`, which trains the baseline t-NN autoencoder with Adam on all weights;
- `ex_mnist_tensor_varpro_regression.py`, which trains the VarPro t-NN autoencoder;
- `tnn/training/autoencoder_trainers.py`, which contains the VarPro autoencoder training definitions, including the warm-up loop, full-training-set feature extraction, tensor least-squares solve, and subsequent Adam updates;
- `plot_loss_curves.py`, which produces the loss comparison figures;
- `plot_reconstructions.py`, which produces the reconstruction figures.

Details for running code and reproducing experiments are contained in the README file in the GitHub Repository above.